

Средства симуляции ЦП и ОС

Лекция №8



Державин Андрей
Шурыгин Антон

Моделирование с использованием трасс

Моделирование на основе трасс

- На прошлых лекциях мы преимущественно обсуждали симуляцию, управляемую исполнением (англ. execution-driven simulation)
- Симуляция на основе трасс (англ. trace-driven simulation) базируется на возможности использования истории дискретных событий для нужд симуляций
- Трасса – истории событий, произошедших в системе за определенный период времени и сохраненные в порядке их возникновения в файл.

История событий в симуляции

- Текстовое представление трассы может иметь следующий вид:

```
time=10  read      addr=0x45df4  result=0x0455
time=14  write     addr=0x35df4  data=0xffff
time=20  interrupt number=10
time=25  port write addr=0x10    data=0xabcd
```

- В данном примере отражены характерные составляющие:
 - time – моменты времени возникновения событий;
 - read, write, interrupt, port write – описание типа события;
 - addr, number – параметры события;
 - result – результаты выполнения.
- Экономия вычислительных ресурсов при повторном исполнении за счет увеличения потребления памяти.

Процесс сбора трасс

- Данные собранные в процессе трассировки должны достаточно точно отражать поведение исходной системы
- Обычно процесс сбора трасс разбивается на несколько стадий

Процесс сбора трасс

- Данные собранные в процессе трассировки должны достаточно точно отражать поведение исходной системы
- Обычно процесс сбора трасс разбивается на несколько стадий
 - Характеризация изучаемого сценария – анализ наиболее интересных фаз для исследования, особенностей изучаемого сценария в целом.

Процесс сбора трасс

- Данные собранные в процессе трассировки должны достаточно точно отражать поведение исходной системы
- Обычно процесс сбора трасс разбивается на несколько стадий
 - Характеризация изучаемого сценария
 - Выбор технологии записи трасс – определение методов детектирования происходящих событий в системе и места хранения собранных трасс (данных).

Процесс сбора трасс

- Данные собранные в процессе трассировки должны достаточно точно отражать поведение исходной системы
- Обычно процесс сбора трасс разбивается на несколько стадий
 - Характеризация изучаемого сценария
 - Выбор технологии записи трасс
 - Выбор интервалов трассировки – определение интервалов в работе сценария, для которых записанные данные будут достаточно точно отражать поведение системы (сэмплирование)

Процесс сбора трасс

- Данные собранные в процессе трассировки должны достаточно точно отражать поведение исходной системы
- Обычно процесс сбора трасс разбивается на несколько стадий
 - Характеризация изучаемого сценария
 - Выбор технологии записи трасс
 - Выбор интервалов трассировки
- Подготовка сценария – модификация изучаемого ПО/оборудования с целью иметь возможность снимать интересующие события/сигналы.

Процесс сбора трасс

- Данные собранные в процессе трассировки должны достаточно точно отражать поведение исходной системы
- Обычно процесс сбора трасс разбивается на несколько стадий
 - Характеризация изучаемого сценария
 - Выбор технологии записи трасс
 - Выбор интервалов трассировки
 - Подготовка сценария
- Валидация – проверка, что полученная трасса верно отражает поведения сценария с допустимой погрешностью

Процесс сбора трасс

- Данные собранные в процессе трассировки должны достаточно точно отражать поведение исходной системы
- Обычно процесс сбора трасс разбивается на несколько стадий
 - Характеризация изучаемого сценария
 - Выбор технологии записи трасс
 - Выбор интервалов трассировки
 - Подготовка сценария
 - Валидация
 - Публикация трассы

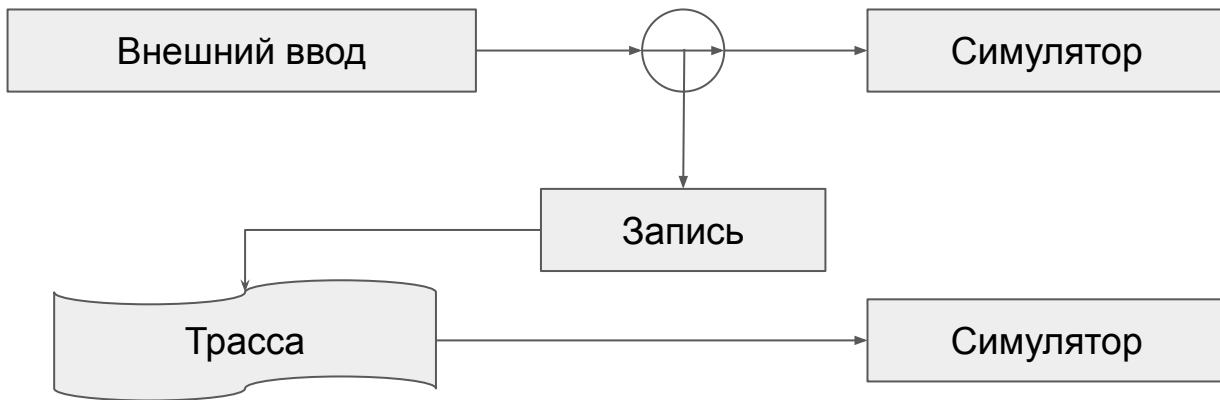
Процесс сбора трасс

- Данные собранные в процессе трассировки должны достаточно точно отражать поведение исходной системы
- Обычно процесс сбора трасс разбивается на несколько стадий
 - Характеризация изучаемого сценария
 - Выбор технологии записи трасс
 - Выбор интервалов трассировки
 - Подготовка сценария
 - Валидация
 - Публикация трассы

Применение трасс

Применение трасс

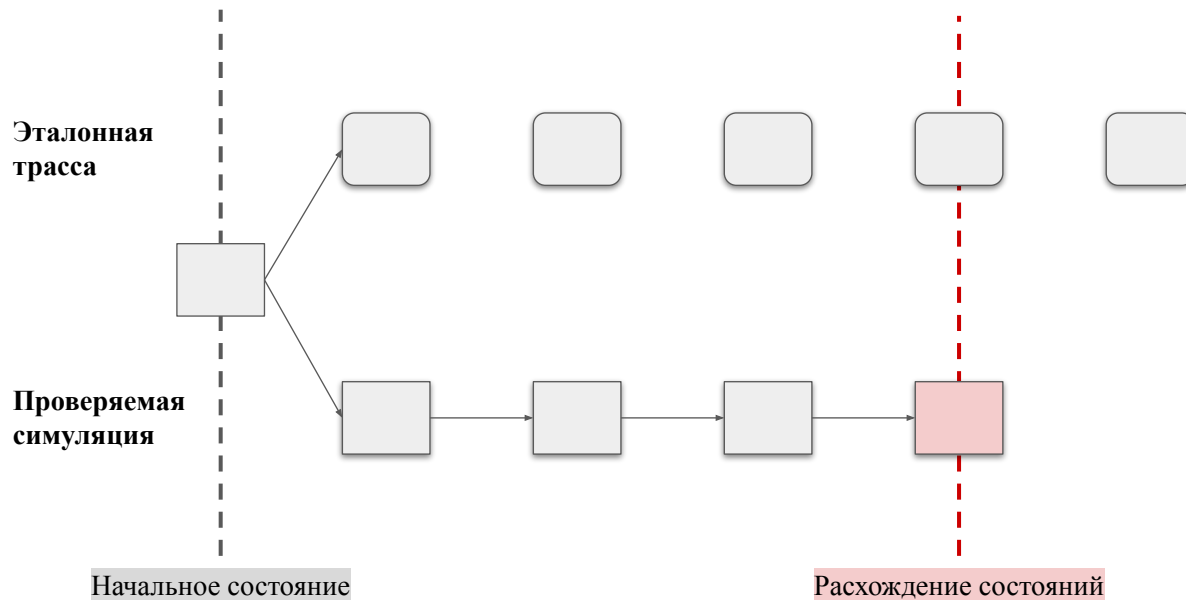
- Рассмотрим области применения симуляции на основе трасс
 - Детерминистический ввод
 - Ввод пользователя с использованием клавиатуры
 - Сетевое взаимодействие



Этапы сбора и использования трассы

Применение трасс

- Рассмотрим области применения симуляции на основе трасс
 - Валидация симулятора



Верификация модели с помощью эталонной трассы на каждом шаге симуляции

Применение трасс

- Моделировать одно и то же явление, получая один и тот же результат?
- **Предложение:** менять характеристики модели, а трассу оставлять неизменной



Применение трасс

- Моделировать одно и то же явление, получая один и тот же результат, сомнительная идея
- Порядок и структура событий в трассе будут одинаковыми при каждом запуске симулятора
- Нет необходимости много раз запускать модель для сборки трассы

Применение трасс

- Изучение поведения программы на процессоре с новой микроархитектурой.

Применение трасс

- Изучение поведения программы на процессоре с новой микроархитектурой.
 - Трасса собирается на существующем процессоре
 - Все доступы в память с результатами отдельных чтений/записей
 - Все внешние и внутренние события (прерывания, исключения)

Применение трасс

- Изучение поведения программы на процессоре с новой микроархитектурой.
 - Трасса собирается на существующем процессоре
 - Трасса подается на модель
 - Результаты используются как история взаимодействия с внешним миром
 - Изменяется внутреннее состояние соответственно трассе, сообщая возникающие задержки

Применение трасс

- Изучение поведения программы на процессоре с новой микроархитектурой.
 - Трасса собирается на существующем процессоре
 - Трасса подаются на модель
- Анализ полученной статистики
 - Нет необходимости писать точную модель оптимизированной аппаратуры
 - Достаточно обладать упрощенной системой задержек

Применение трасс

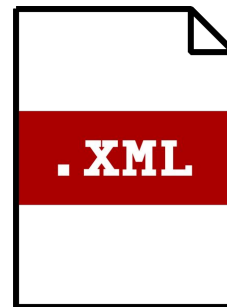
- Изучение поведения программы на процессоре с новой микроархитектурой.
 - Трасса собирается на существующем процессоре
 - Трасса подаются на модель
 - Анализ полученной статистики
- При таком подходе нет необходимости иметь доступ ни к исходному коду, ни к исполняемым файлам изучаемого приложения
- Удобно изучать закрытые или ограниченные в распространении приложения, работая лишь с трассой

Форматы хранения трасс

- Обычно трассы хранятся прямо на жестком диске в файле.
- Файлы разделяют на две категории:
 - Текстовый формат
 - Бинарный формат

Форматы хранения трасс

- Обычно трассы хранятся прямо на жестком диске в файле.
- Файлы разделяют на две категории:
 - Текстовый формат
 - XML, JSON, CSV
 - Содержимое файла без труда может быть изучено человеком
 - Бинарный формат



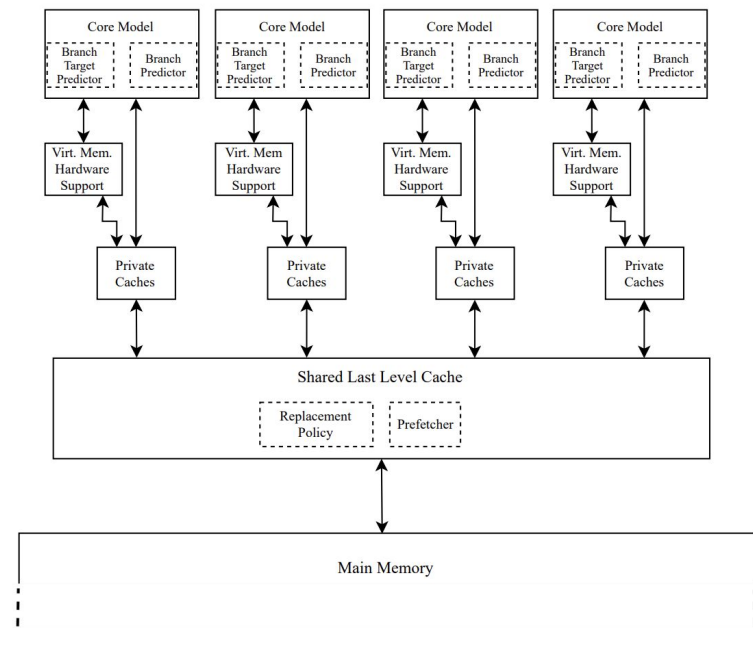
Форматы хранения трасс

- Обычно трассы хранятся прямо на жестком диске в файле.
- Файлы разделяют на две категории:
 - Текстовый формат
 - Бинарный формат
 - Обработка двоичного формата для симулятора более удобная и быстрая
 - Общепринятого формата нет

Практическая часть

От слов — к делу

- [ChampSim](#) — симулятор с открытым исходным кодом, основанный на трассировке, поддерживаемый Техасским университетом A&M
- Был разработан как платформа для соревнований по микроархитектуре
- Активно используется для разработки множества современных политик замены кэша и префетч-политик



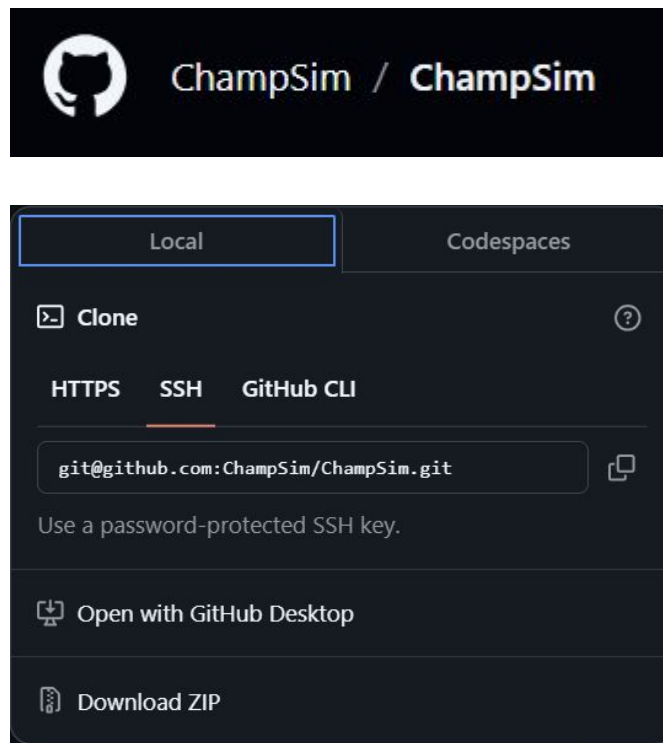
От слов – к делу

- Почему именно ChampSim?
 - Быстрый trace-based симулятор
 - Можно проводить исследования без глубокого погружения в микроархитектуру
 - Модульность
 - Разнообразие сред моделирования



От слов – к делу

- Внимательно читаем [README](#) и не делаем ничего лишнего:
 - Клонировем проект



От слов – к делу

- Внимательно читаем [README](#) и не делаем ничего лишнего:
 - Клонировем проект
 - Устанавливаем сторонние зависимости

```
git submodule update --init  
vcpkg/bootstrap-vcpkg.sh  
vcpkg/vcpkg install
```

От слов – к делу

- Внимательно читаем [README](#) и не делаем ничего лишнего:
 - Клонировем проект
 - Устанавливаем сторонние зависимости
 - Собираем проект

```
$ ./config.sh <configuration file>  
$ make -j <jobs>
```

От слов – к делу

- Внимательно читаем [README](#) и не делаем ничего лишнего:
 - Клонировуем проект
 - Устанавливаем сторонние зависимости
 - Собираем проект
- ChampSim при сборке требует конфигурационный файл.
- Все параметры в файле являются опциональными

```
1  {
2      "executable_name": "champsim",
3      "block_size": 64,
4      "page_size": 4096,
5      "heartbeat_frequency": 10000000,
6      "num_cores": 1,
7
8      "ooo_cpu": [
9          {
10             "frequency": 4000,
11             "ifetch_buffer_size": 64,
12             "decode_buffer_size": 32,
13             "dispatch_buffer_size": 32,
14             "rob_size": 352,
15             "lq_size": 128,
16             "sq_size": 72,
17             "fetch_width": 6,
18             "decode_width": 6,
19             "dispatch_width": 6,
20             "execute_width": 4,
21             "lq_width": 2,
22             "sq_width": 2,
23             "retire_width": 5,
24             "mispredict_penalty": 1,
25             "scheduler_size": 128,
26             "decode_latency": 1,
27             "dispatch_latency": 1,
28             "schedule_latency": 0,
29             "execute_latency": 0,
30             "branch_predictor": "bimodal",
31             "btb": "basic_btb"
32         }
33     ],
```

Конфигурационный файл champs config.json

От слов — к делу



- Затем переходим в [хранилище](#) и скачиваем трассы с тестов производительности SPEC CPU 2017
 - Например, скачаем самую первую, `400.perlbench-41B.champsimtrace.xz`

```
.  
— 400.perlbench-41B.champsimtrace.xz  
— 400.perlbench-50B.champsimtrace.xz  
— 401.bzip2-226B.champsimtrace.xz  
— 401.bzip2-277B.champsimtrace.xz  
— 401.bzip2-38B.champsimtrace.xz  
— 401.bzip2-7B.champsimtrace.xz  
— 403.gcc-16B.champsimtrace.xz  
— 403.gcc-17B.champsimtrace.xz  
— 403.gcc-48B.champsimtrace.xz  
— 410.bwaves-1963B.champsimtrace.xz  
— 410.bwaves-2097B.champsimtrace.xz  
— 410.bwaves-945B.champsimtrace.xz  
— 416.gamess-875B.champsimtrace.xz  
— 429.mcf-184B.champsimtrace.xz  
— 429.mcf-192B.champsimtrace.xz  
— 429.mcf-217B.champsimtrace.xz  
— 429.mcf-22B.champsimtrace.xz  
— 429.mcf-51B.champsimtrace.xz
```

Хранилище трасс SPEC CPU 2017

От слов — к делу

- Тут вы должны задаться вопросом...



Вернемся к применению трасс

- Говоря о закрытых приложениях и “обезличенных” трассах, мы должны понимать, что:
 - В трассе отражены лишь внешние события
 - Изменения во внутреннем состоянии отслеживаются моделью
- Рассмотрим более конкретный анализ – изучение производительности кэшей.
 - Трасса содержит информацию о последовательности, типах, адресах доступов
 - Количество линий, емкость и прочие микроархитектурные аспекты определяются моделью

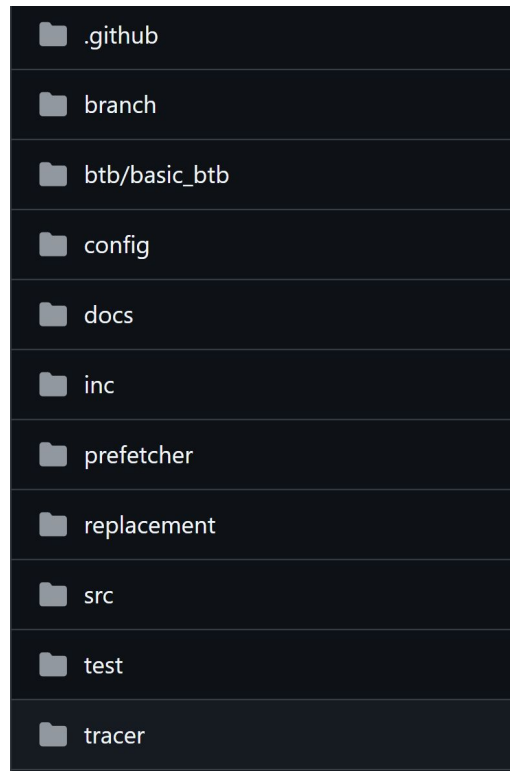
От слов – к делу

- Вспоминаем про модульность и простоту конфигурации ChampSim.



От слов – к делу

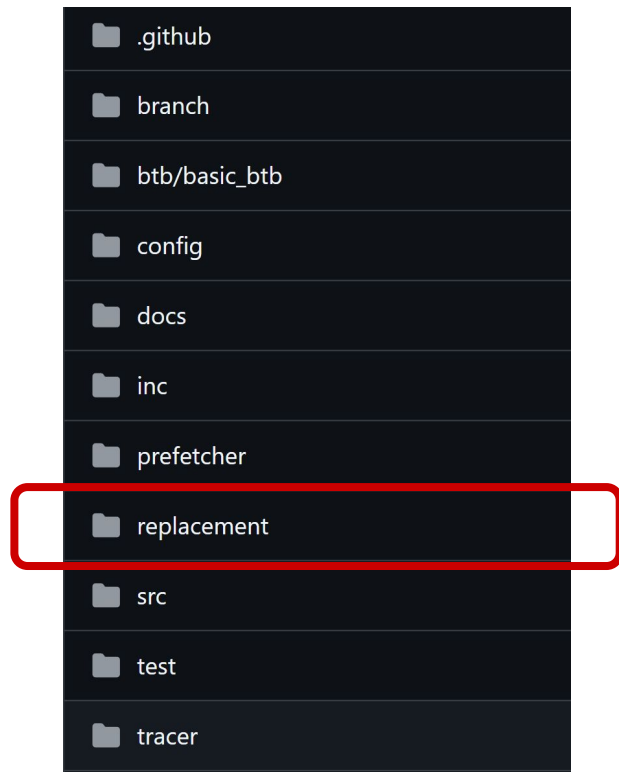
- Вспоминаем про модульность и простоту конфигурации ChampSim
- Смотрим на структуру проекта и видим:
 - Branch Predictors
 - Branch Target Buffers
 - Memory Prefetchers
 - Replacement Policies



Структура проекта ChampSim

От слов – к делу

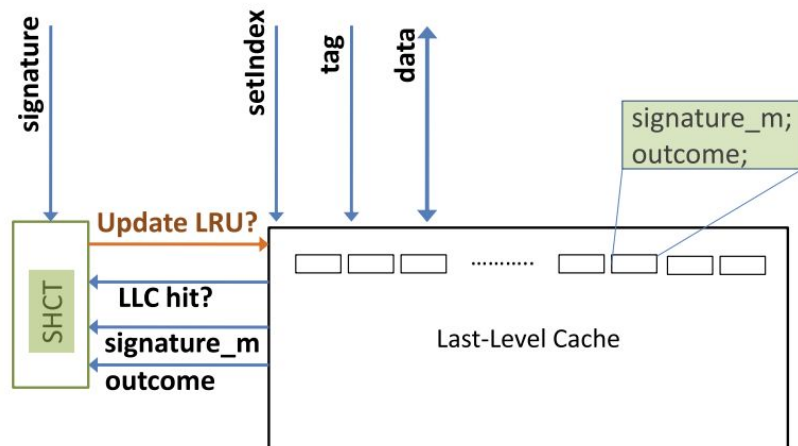
- Вспоминаем про модульность и простоту конфигурации ChampSim
- Смотрим на структуру проекта и видим:
 - Branch Predictors
 - Branch Target Buffers
 - Memory Prefetchers
 - **Replacement Policies**



Структура проекта ChampSim

Встроенные политики замещения

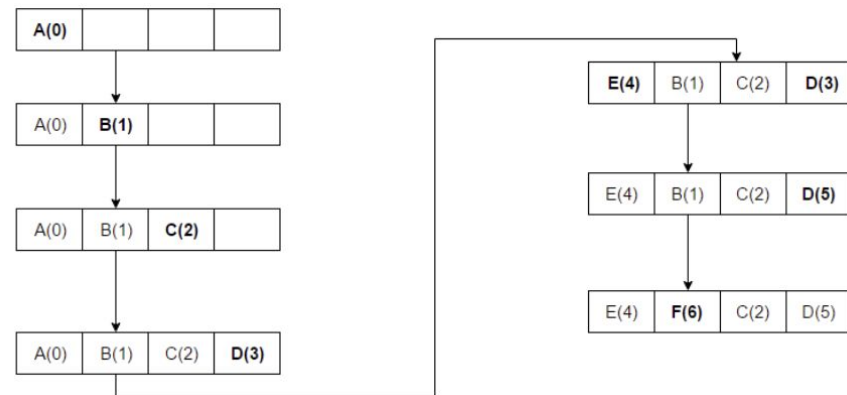
- На мастере нам сразу доступны четыре политики
 - LRU – Least Recently Used
 - SRRIP – Static Re-Reference Interval Prediction
 - DRRIP – Dynamic Re-Reference Interval Prediction
 - SHIP – Signature-Based Hit Prediction



Архитектура SHiP

Встроенные политики замещения

- На мастере нам сразу доступны четыре политики
 - **LRU – Least Recently Used**
 - SRRIP – Static Re-Reference Interval Prediction
 - DRRIP – Dynamic Re-Reference Interval Prediction
 - SHIP – Signature-Based Hit Prediction



LRU

Объявление новой политики замещения

- Модуль политики замещения кэшей должен определять четыре функции:
 - `void CACHE::initialize_replacement ();`
 - `uint32_t CACHE::find_victim (...);`
 - `void CACHE::update_replacement_state (...);`
 - `void CACHE::replacement_final_stats ();`
- Подробнее можно ознакомиться в документации ChampSim.

От слов – к делу.

- В дополнение к LRU определим еще две простые политики замещения:
 - LFU – Least Frequently Used
 - MRU – Most Recently Used
- Давайте проведем небольшую исследовательскую работу и изучим пространство конфигураций модели ChampSim



```
./champsim -w 5000000 -i 20000000 --json <path/to/400.perlbench-41b.json>  
<path/to/400.perlbench-41B.champsimtrace.xz>
```

Золотое правило computer science

- Производительность

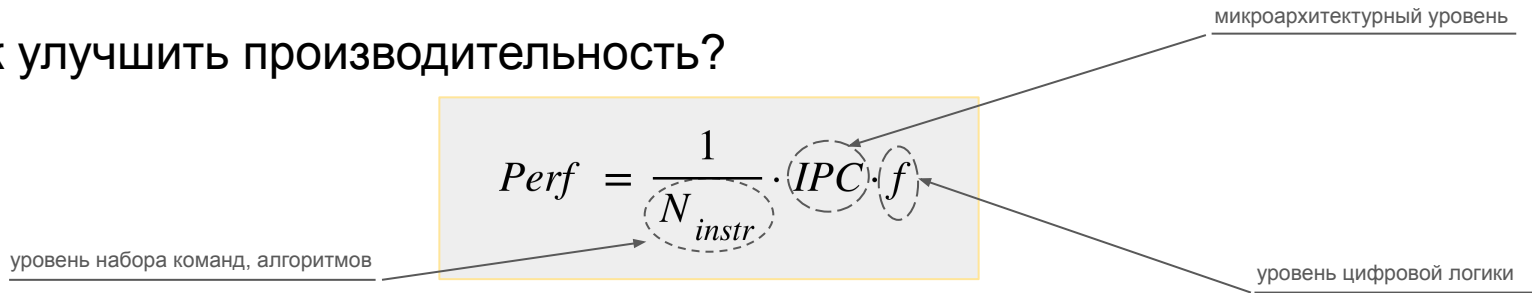
$$Perf = \frac{1}{Time} = \frac{1}{N_{instr} \cdot CPI \cdot T_{cycle}} = \frac{1}{N_{instr}} \cdot IPC \cdot f$$

- Время исполнения

$$Time = T_{cycle} \cdot Cycles\ Num = N_{instr} \cdot CPI \cdot T_{cycle}$$

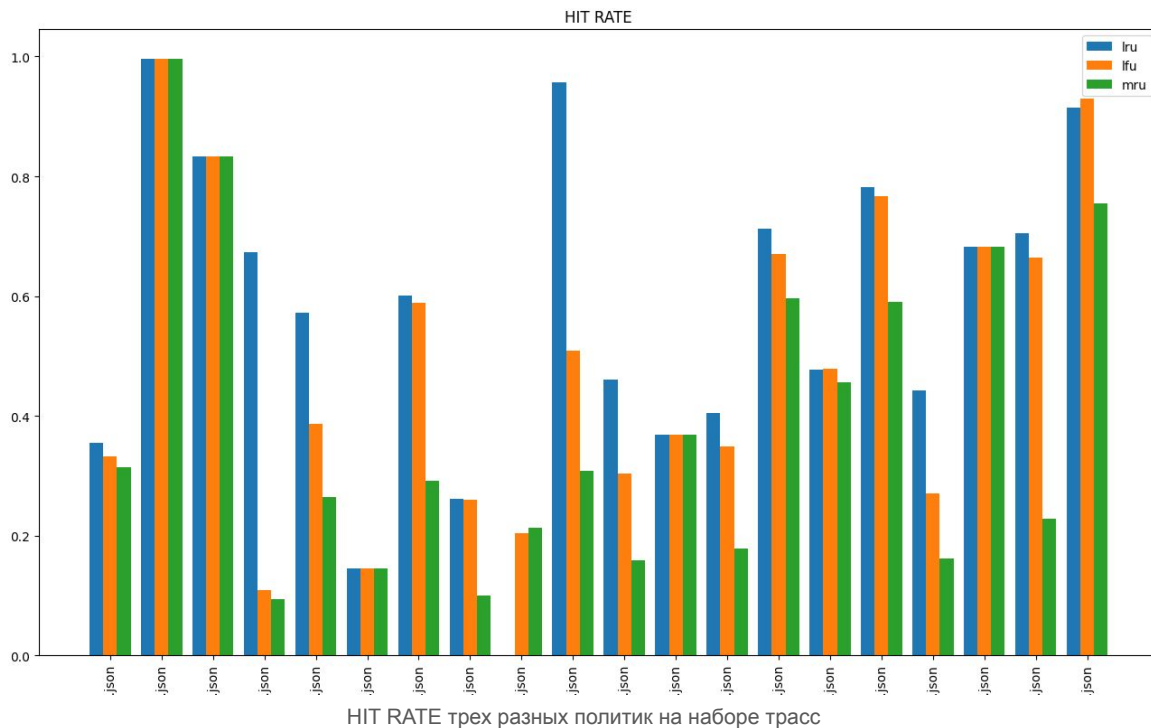
1. **IPC** - среднее количество инструкций, выполняемых за каждый такт
2. **CPI** - величина, обратная IPC.
3. **f** - тактовая частота

- Как улучшить производительность?



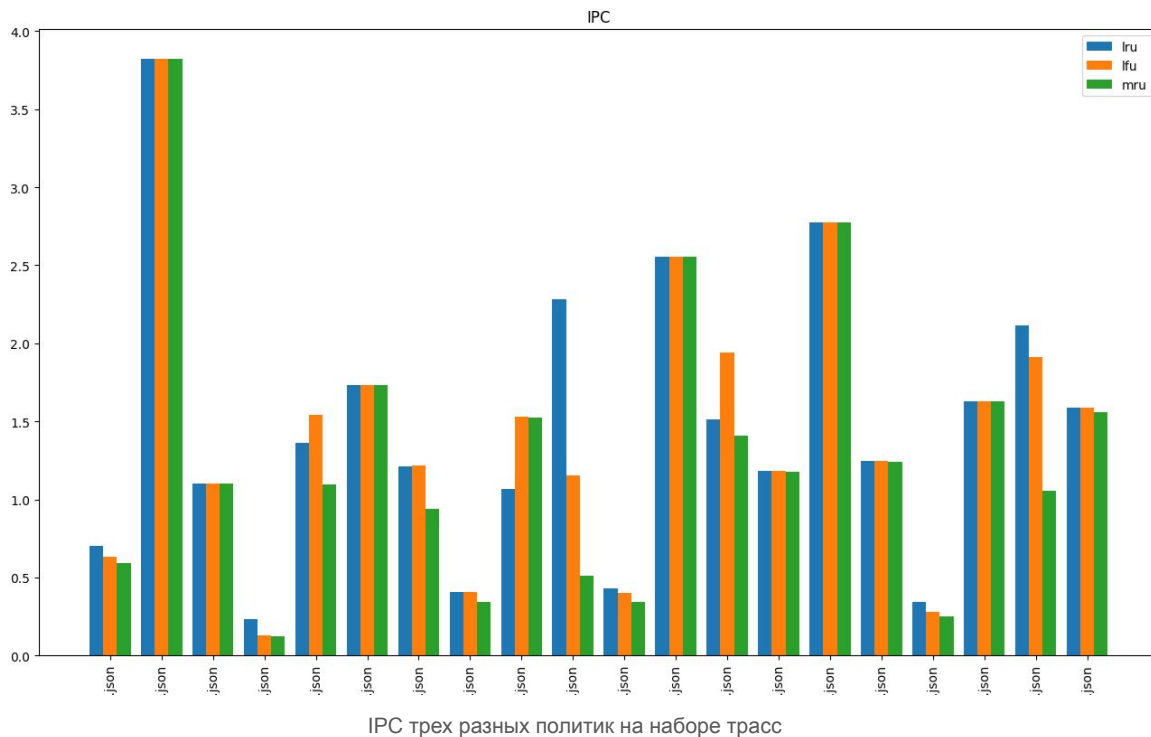
Анализ результатов

- Из выходного JSON файла с результатами подсчитываем HIT RATE по L2C

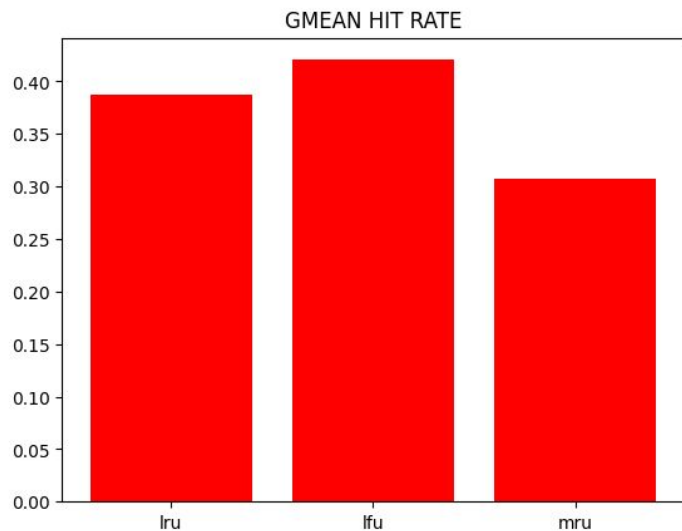


Анализ результатов

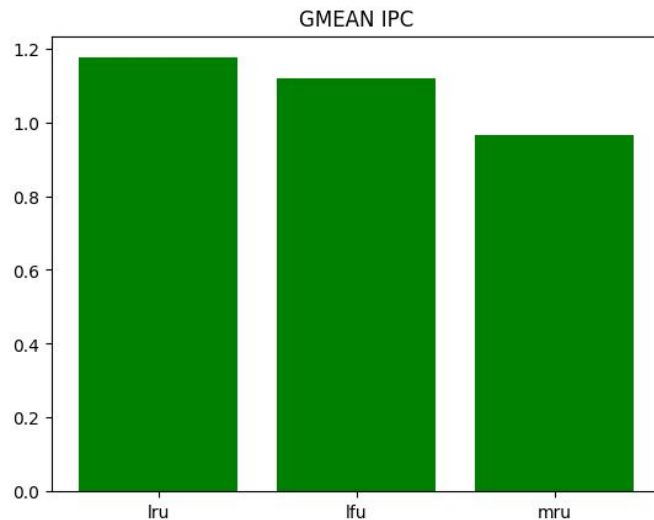
- Из выходного JSON файла с результатами собираем статистику IPC (instructions per cycle)



Анализ результатов



Среднее геометрическое HIT RATE для трех разных политик

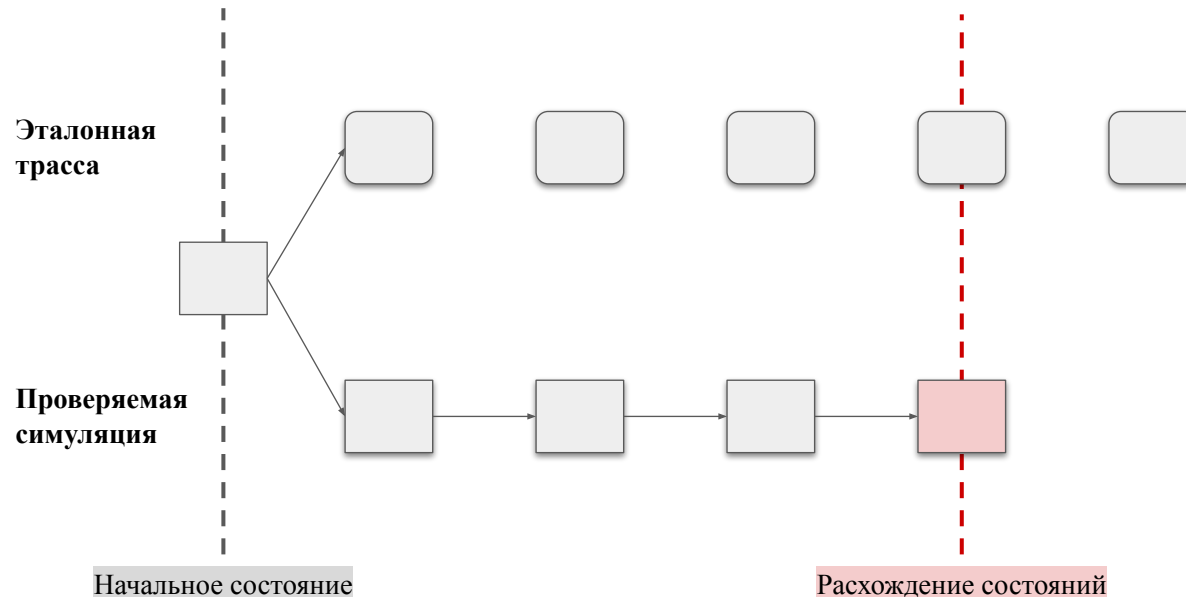


Среднее геометрическое IPC для трех разных политик

ДЗ 2-1

ДЗ 2-1

- Сгенерируйте косимуляционный модуль, позволяющий проводить валидацию симулятора с эталонной моделью (напр., QEMU)



Ссылки

1. [Simulation Book, Grigoriy Rechistov, 2023](#)
2. [Precise and Accurate Processor Simulation](#)
3. [The Championship Simulator: Architectural Simulation for Education and Competition](#)
4. [SHiP: Signature-based Hit Predictor for High Performance Caching](#)